KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 4 Laboratuvar Föyü

React I: Bileşenler — Hafta 3'ün Elle DOM Kodunu Bileşenlere Dönüştürün

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Tahmini süre: 2 saat
Gereksinimler: Node.js 24 LTS kurulu bir bilgisayar (Hafta 3'te kurdunuz; daha yeni bir güncel sürüm — ör. 26 — de çalışır, ama ders boyunca kararlılık için 24 LTS önerilir), bir metin editörü (VS Code önerilir), güncel bir tarayıcı (Chrome/Firefox/Edge), Hafta 3'te tamamladığınız `istemci` klasörü — npm paketlerini indirmek için internet bağlantısı gerekir

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim — Neden Önemli?

Hafta 3'te bir koltuğa tıklandığında hem koltuğun kendi rengini (`classList.toggle`) hem sayfanın başka bir yerindeki "seçim özeti" metnini ayrı ayrı, elle güncellemeniz gerekiyordu — bu iki güncellemeden birini unutsaydınız ekran, gerçek durumdan farklı bir şey gösterirdi. Bugün iki saat sonra, aynı koltuk/etkinlik verisiyle, tıklama davranışını yalnızca tek bir yerde — `Koltuk` bileşeninin kendi içinde — tanımlayacak, ekranı güncellemeyi React'e bırakacaksınız. `querySelector`/`addEventListener`/ `classList` ile elle DOM yönetiminin yerini bugün **bileşenler** alacak — Etkinlik/Bilet Sisteminin ön yüzü artık HTML'e elle yazılan ya da DOM API'siyle elle üretilen öğeler değil, TSX bileşenleri olacak.

1 • Ön-öğretim — Ön Bilgi

Bu föy Hafta 4 ders notunun dört ünitesine dayanır: Ünite 1 (Neden Bileşen? React'e Giriş ve TSX), Ünite 2 (props ve Tiplenmiş Bileşen), Ünite 3 (state ve Olaylar) ve Ünite 4 (Liste ve Koşullu Render). Hafta 3'ün `istemci` klasöründeki `tipler.ts`/`veri.ts` dosyalarını (Etkinlik/Koltuk/Bilet tipleri, sabit diziler) ve Vite/npm araç zinciri bilginizi hazır bilgi olarak kullanacaksınız — bu iki dosya bugün değişmeden yeni projenize kopyalanacak. YG I/II'den C/C++'ın `const` disiplinini ve bir fonksiyona parametre geçirme alışkanlığınızı da köprü olarak kullanacaksınız: `props` bir fonksiyon parametresine, `useState`'in değişmezlik kuralı ise `const` bir değişkeni değiştirememe disiplinine benzer. Bu hafta `fetch`, gerçek bir API, formlar, `useEffect` ve React Router henüz yoktur — yalnızca sabit (mock) veriyle çalışacaksınız (Hf.5–7'nin konusudur).

Föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi kendi `istemci-react` projenize **elle yazarak** tamamlayın, sonra editörünüzde/tarayıcınızda gördüğünüzü *Beklenen sonuç* kutusuyla karşılaştırın. Adımlardaki kod kutuları `kod_ornekleri/hafta04/istemci/` klasöründeki referans dosyalardan gelir — bunlar hedef/"doğru cevap" niteliğindedir; siz bu kodu kopyala-yapıştır yapmak yerine kendi dosyalarınıza yazarsınız, sonucu editörde/tarayıcıda karşılaştırırsınız. Adımlar ders notunun dört ünitesiyle aynı sırada ilerler; her bölümün başında ilgili üniteye bir atıf bulacaksınız. Birkaç adımda bir gelen *Kontrol Noktası*, o ana kadar yazdıklarınızı bir arada toparlamanıza yarar.

1 Neden Bileşen? React'e Giriş ve TSX

(bkz. Ders notu Hafta 4, Ünite 1 — Neden Bileşen? React'e Giriş ve TSX.)

Adım 1: `istemci-react` klasörünü `react-ts` şablonuyla oluşturun

Ne yapacağız? Hafta 3'ün `istemci` klasörünün yanında (aynı üst klasörde), `npm create vite@latest` komutuyla bu kez **React + TypeScript** (`react-ts`) şablonlu yeni bir proje oluşturacak, projeyi `istemci-react` adıyla kaydedecek ve bağımlılıklarını indireceksiniz.

Neden? Hafta 3'ün `istemci` klasörüne dokunmayacaksınız — o proje bu haftanın `tipler.ts/veri.ts` dosyalarının kaynağı olarak kalacak. `react-ts` şablonu, `vanilla-ts`'in aksine `react/react-dom` bağımlılıklarını ve `.tsx` uzantılı dosyaları önceden hazırlar; Vite'ın kendisi (derleme/paketleme mantığı) değişmez, yalnızca üstüne React'in JSX dönüşümünü uygulayan bir eklenti gelir.

Terminal

```
$ npm create vite@latest istemci-react -- --template react-ts
$ cd istemci-react
$ npm install
```

Beklenen sonuç: `istemci-react` klasöründe `index.html`, `src/main.tsx`, `src/App.tsx`, `vite.config.ts` dosyaları ve `npm install` sonrası bir `node_modules` klasörü görünür.

Beklenen Çıktı

```
added 27 packages, and audited 28 packages in 28s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

İpucu

Şablon ayrıca `App.css`, `index.css`, `src/assets/react.svg`, `public/vite.svg`, `.oxlintrc.json` gibi Vite'ın kendi tanıtım dosyalarını da oluşturur — bunlara dokunmanıza gerek yok, ilerleyen adımlarda kendi `index.html/main.tsx/App.tsx` içeriğinizle üzerlerine yazacaksınız.

Adım 2: `package.json` ve `vite.config.ts` dosyalarını inceleyin

Ne yapacağız? Oluşan `package.json` dosyanızdaki `dependencies` (`react`, `react-dom`) ve `devDependencies` (`@types/react`, `@types/react-dom`, `@vitejs/plugin-react`) alanlarını inceleyecek; `vite.config.ts` dosyasına şablonun otomatik eklediği `@vitejs/plugin-react` eklentisini göreceksiniz; referans dosyadaki `engines` alanını kendi `package.json`'ınıza da ekleyeceksiniz.

Neden? `react/react-dom` çalışma zamanında (`dependencies`) gerekirken, TypeScript'in React tiplerini tanıması için `@types/react/@types/react-dom` yalnızca geliştirme sırasında (`devDependencies`) gerekir — Hafta 3'ün `typescript/vite` ayrımıyla aynı mantık. `@vitejs/plugin-react` eklentisi, Vite'a `.tsx` dosyalarındaki JSX'i React'in kendi fonksiyon çağrılarına dönüştürmesi gerektiğini söyler — bu eklenti olmadan Vite bir `<div>` ifadesini anlamlandıramaz.

Adım 2 (devamı)

```
{
  "name": "istemci",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "engines": {
    "node": "^24.13.0 || >=24.13.0"
  },
  "scripts": {
    "dev": "vite",
    "build": "tsc -b && vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8"
  },
  "devDependencies": {
    "@types/node": "^24.13.3",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "typescript": "^7.0.2",
    "vite": "^8.2.2"
  }
}
```

```
import react from '@vitejs/plugin-react';
import { defineConfig } from 'vite';

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
});
```

Beklenen sonuç: Kendi `vite.config.ts`'inizde `plugins: [react()]` satırı zaten hazır gelir — `npm create vite@latest`'in `react-ts` şablonu bunu sizin için otomatik ekler. Referans dosyadaki `react/typescript/vite` sürümleriyle kendi `package.json`'ınızdaki sürümler birebir aynı olmayabilir — bu normaldir; önemli olan `scripts` alanının aynı üç komutu (`dev`, `build`, `preview`) içermesi ve `react`/`react-dom`'un `dependencies` altında görünmesidir.

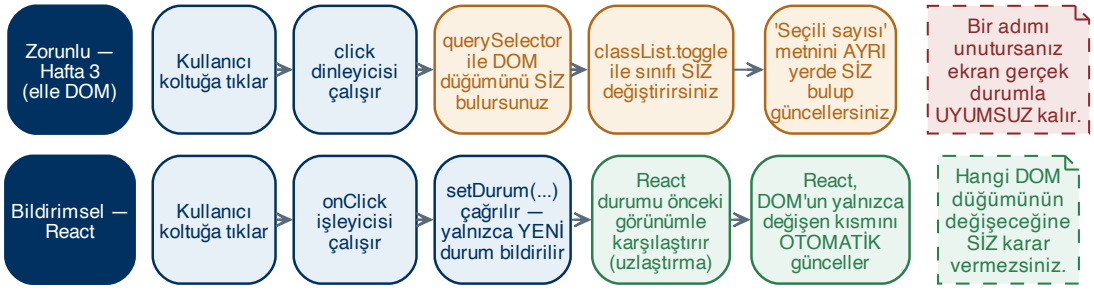
Adım 3: Geliştirme sunucusunu başlatıp bir JSX ifadesini değiştirin

Ne yapacağız? `npm run dev` komutuyla geliştirme sunucusunu başlatıp tarayıcıda `http://localhost:5173` adresini açacaksınız; ardından `src/App.tsx` dosyasındaki süslü parantez (`{ }`) içindeki bir ifadeyi (ör. sayaç metnini ya da başlığı) değiştirip kaydedeceksiniz.

Neden? Zorunlu (*imperative*) yaklaşımda "DOM'u nasıl güncelleyeceğinizi" adım adım siz yazarsınız (Hafta 3'ün `querySelector` → bul → `classList.toggle` zinciri); bildirimsel (*declarative*) yaklaşımda yalnızca "durum ne ise görünüm ne olmalı" tanımlarsınız, güncellemenin nasıl yapılacağına React karar verir. `App.tsx`'teki etiketler HTML'e benzer ama aslında JavaScript fonksiyon çağrılarına derlenen bir sözdizimidir (JSX/TSX); süslü parantez içine yalnızca bir değer üreten bir ifade yazabilirsiniz (`1 + 1`, bir değişken, bir fonksiyon çağrısı) — `if`/`for` gibi bir deyim yazamazsınız. Kaydettiğinizde Vite'ın anlık güncelleme

Adım 3 (devamı)

(HMR) özelliği sayfayı tamamen yeniden yüklemeden yalnız değişen kısmı günceller — bu, React'in her güncellemede önce bir taslak (sanal DOM) hesaplayıp eski taslakla karşılaştırdığı (uzlaştırma) ve yalnızca farkı gerçek DOM'a uyguladığı fikrinin ilk somut göstergesidir.



Her iki akış da AYNI kullanıcı tıklamasından başlar; fark, görünümü kimin güncellediğidir.

Terminal

```
$ npm run dev
```

Beklenen sonuç: Terminalde aşağıdaki gibi bir başlangıç bildirimi görünür; tarayıcıda Vite'ın varsayılan React karşılama sayfası (dönen React logosu, bir sayaç düğmesi) açılır. `App.tsx`'te yaptığınız değişiklik, sayfayı yenilemenize gerek kalmadan anında yansır.

Beklenen Çıktı

```
  VITE v8.2.2  ready in 110 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
```

Ortak Hata / Tuzak

Bir bileşenin **tek bir kök öğe** döndürmesi gerektiğini unutup iki kardeş öğeyi doğrudan döndürmek (`return <h1>...</h1> <p>...</p>;`) derleme hatasına yol açar. İki öğeyi ortak bir `<div>` ile sarmak ya da ekstra bir DOM düğümü oluşturmak istemiyorsanız görünmez bir Fragment (`<>...</>`) kullanmak bu sorunu çözer.

Kontrol Noktası — devam etmeden doğrulayın

Tarayıcıda `npm run dev` ile açılan sayfada bir JSX ifadesini değiştirip HMR'ın anlık güncellediğini gördüyseniz, bildirimsel modelin ilk somut örneğini yaşamışsınız demektir — artık kendi bileşenlerinizi yazmaya başlayabilirsiniz.

2 props ve Tiplenmiş Bileşen

(bkz. Ders notu Hafta 4, Ünite 2 — props ve Tiplenmiş Bileşen.)

Adım 4: `tipler.ts` ve `veri.ts` dosyalarını Hafta 3'ten aynen kopyalayın

Ne yapacağız? `istemci-react/src` klasörünüzde, Hafta 3'ün `istemci/src` klasöründeki `tipler.ts` ve `veri.ts` dosyalarını aynen kopyalayacak, `diff` ile iki dosyanın birebir aynı olduğunu doğrulayacaksınız.

Neden? Bu hafta sunucu (FastAPI) henüz yok — veri modeliniz değişmedi. `Etkinlik`/`Koltuk`/`Bilet` tipleri ve sabit diziler, Hafta 3'te tanımladığınız hâliyle bugünden itibaren React bileşenlerinin `props` tipleri olarak kullanılmaya devam edecek.

Terminal

```
$ cp ../istemci/src/tipler.ts src/tipler.ts
$ cp ../istemci/src/veri.ts src/veri.ts
$ diff ../istemci/src/tipler.ts src/tipler.ts
$ diff ../istemci/src/veri.ts src/veri.ts
```

Beklenen sonuç: Her iki `diff` komutu da hiçbir satır yazdırmadan biter — bu, iki dosyanın birebir aynı olduğu anlamına gelir (Hafta 3'teki `tsc --noEmit`'in hatasız bitmesi gibi, "sessizlik" burada da iyi bir işarettir).

```ts
export interface Etkinlik {
  id: number;
  baslik: string;
  tarih: string; // ISO 8601 tarih dizesi, ör. "2026-11-14"
  mekan: string;
  kapasite: number;
  biletFiyati: number; // TL cinsinden birim koltuk fiyatı
}
```

```ts
export type KoltukDurumu = 'bos' | 'secili' | 'dolu';

export interface Koltuk {
  readonly id: number;
  readonly etkinlikId: number;
  sira: number;
  koltukNo: number;
  durum: KoltukDurumu;
}
```

> **İpucu**
>
> Aynı dosyada `BiletDurumu`/`Bilet` ve generic `Liste<T>` tipleri de bulunur — bu hafta hiçbir bileşen `Bilet` ya da `Liste<T>` kullanmaz (koltuk seçme formu ve sepet Hf.5'in konusudur), bu yüzden onları burada tekrar göstermiyoruz; dosyanız Hafta 3'teki hâliyle aynen kalmalıdır.

Adım 5: `index.html`'i inceleyip `style.css` oluşturun, `main.tsx`'i bağlayın

Ne yapacağız? `index.html` dosyanızın gövdesinde `react-ts` şablonunun zaten hazırladığı `<div id="root">` kökünü ve `main.tsx`'e bağlanan `<script type="module">` etiketini inceleyeceksiniz; ardından `src` klasörünüzde `style.css` adlı yeni bir dosya oluşturup içine Hafta 3'ün stillerinden uyarlanmış `.bolum` başlık stilini ve `.koltuk.secili`/`.koltuk.dolu` durum stillerini yazacak, son olarak `src/main.tsx`'in en üstündeki `import './index.css';` satırını `import './style.css';` olarak değiştireceksiniz (dosyanın geri kalanına henüz do-

Adım 5 (devamı)

kunmayın).

Neden? `react-ts` şablonu `index.html`'i zaten Vite'a uygun hazırlar — Hafta 3'te elle yazdığınız `<div id="root">` kökü bu kez şablonun kendisinden gelir. `.koltuk.secili`/`.koltuk.dolu` sınıfları, birazdan yazacağınız `Koltuk` bileşeninin `className` ifadesiyle açıp kapatacağı görsel durumlardır — Hafta 3'te bunu `classList.toggle` ile elle yapıyordunuz.

```
<div id="root"></div>
<script type="module" src="/src/main.tsx"></script>
```

```
.bolum {
  margin: 1.5rem 0;
}

.bolum h2 {
  border-bottom: 2px solid var(--renk-mavi);
  padding-bottom: 0.25rem;
}
```

```
.koltuk.secili {
  background-color: var(--renk-yesil);
  color: #FFFFFF;
  border-color: var(--renk-yesil);
}

.koltuk.dolu {
  background-color: var(--renk-acikgri);
  border-color: var(--renk-gri);
  color: var(--renk-gri);
  cursor: not-allowed;
}
```

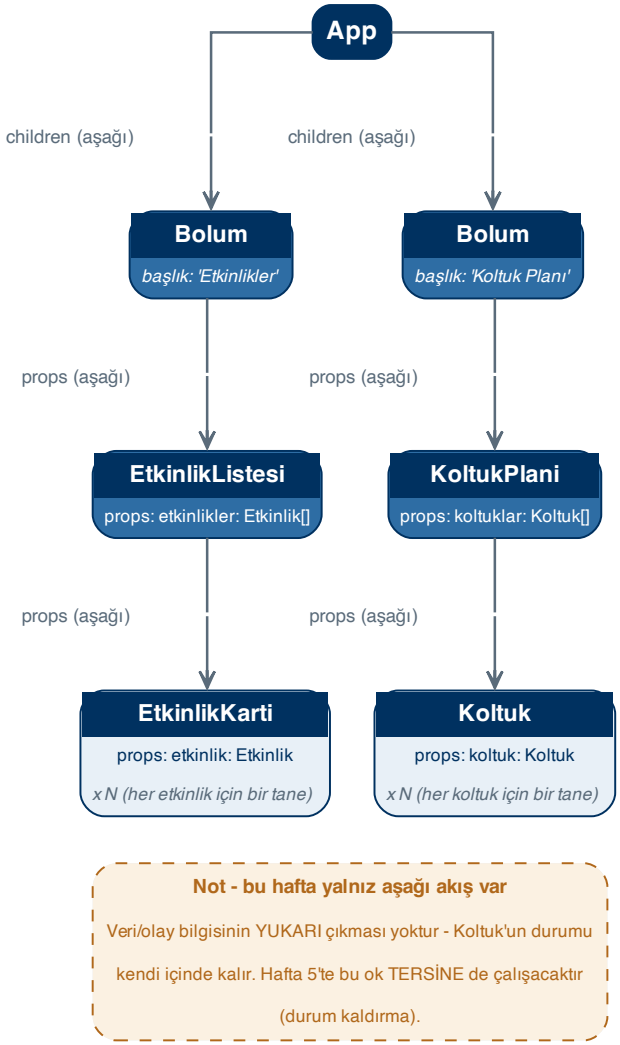
Beklenen sonuç: `npm run dev` çalışırken tarayıcıyı yenilediğinizde sayfanın yazı tipi ve rengi değişir (yeni `style.css`'iniz devrede) — ekranda hâlâ Vite'ın varsayılan sayaç düğmesi görünür, bunu birazdan değiştireceksiniz.

Adım 6: `EtkinlikKarti` bileşenini `props` ile yazın

Ne yapacağız? `src/bilesenler` klasörünü oluşturup içine `EtkinlikKarti.tsx` dosyasını ekleyecek; `etkinlik: Etkinlik` alanı taşıyan `EtkinlikKartiProps` arayüzünü ve bu prop'u alıp başlık/tarih/mekân/fiyatı gösteren `EtkinlikKarti` fonksiyon bileşenini yazacaksınız.

Neden? Bir React bileşeni, JSX döndüren sıradan bir TypeScript fonksiyonudur — ama büyük harfle başlamalıdır, çünkü React küçük harfle başlayan adları bir HTML etiketi sanır. `props`, bir bileşene dışarıdan veri geçirmenin tek yoludur; bir fonksiyona parametre geçirmeye benzer ama salt okunur kabul edilir — bileşen kendisine gelen `props`'u doğrudan değiştirmez. Fiyat 0 ise üçlü operatörle (`? :`) "Ücretsiz" rozeti gösterme, bu haftanın ilk koşullu render örneğidir.

Adım 6 (devamı)




```tsx
interface EtkinlikKartiProps {
  etkinlik: Etkinlik;
}
```

```tsx
// Hf.2'nin article.etkinlik-karti HTML yapısının React karşılığı;
// biletFiyati 0 ise üçlü operatörle "Ücretsiz" rozeti gösterilir.
export function EtkinlikKarti({ etkinlik }: EtkinlikKartiProps) {
  return (
    <article className="etkinlik-karti">
      <h3>{etkinlik.baslik}</h3>
      <p>
        {etkinlik.tarih} · {etkinlik.mekan}
      </p>
      <p>
        Kapasite: {etkinlik.kapasite} ·{' '}
        {etkinlik.biletFiyati === 0 ? (
          <span className="rozet-ucretsiz">Ücretsiz</span>
        ) : (
          `Bilet: ${etkinlik.biletFiyati} TL`
        )}
      </p>
```

Adım 6 (devamı)

```
    </article>
  );
}
```

Beklenen sonuç: Dosyayı kaydettiğinizde editörünüzde (VS Code) hiçbir kırmızı çizgi görünmez — bu bileşeni henüz `App.tsx`'te çağırmadınız, tarayıcıda hâlâ görünmeyecektir.

Ortak Hata / Tuzak

Bileşen adını küçük harfle yazmak (`etkinlikKarti` gibi) en sık yapılan hatalardan biridir — React bunu bir HTML etiketi (`<etkinlikkarti>`) sanır, bileşeniniz hiçbir şey render etmeden sessizce ya da tarayıcı konsoluna farklı bir uyarıyla başarısız olur. Bileşen adları her zaman PascalCase ile (büyük harfle) başlamalıdır: `EtkinlikKarti`.

Adım 7: `Bolum` (`children`) sarmalayıcı bileşenini yazın

Ne yapacağız? `src/bilesenler/Bolum.tsx` dosyasını oluşturup `baslik: string` ve `children: ReactNode` alanlarını taşıyan `BolumProps` arayüzünü, ardından bu ikisini bir `<section>` içinde birleştiren `Bolum` bileşenini yazacaksınız.

Neden? Bir bileşenin açılış/kapanış etiketleri arasına yazılan JSX, otomatik olarak `props.children` olarak gelir — bu, bir bileşeni "sarmalayıcı" (*wrapper*) olarak kullanmayı mümkün kılar. `Bolum`, kendi içine hangi JSX'in konacağını bilmeden yalnızca bir başlık + çerçeve sağlar; "Etkinlikler" ve "Koltuk Planı" bölümlerinin ikisi de aynı `Bolum` bileşenini farklı `children` ile kullanacaktır.

```
// children örneği: Bolum, açılış/kapanış etiketleri arasına konan
// HERHANGİ bir JSX'i props.children olarak alıp bir başlığın altında
// gösteren küçük bir sarmalayıcı bileşendir.
interface BolumProps {
  baslik: string;
  children: ReactNode;
}

export function Bolum({ baslik, children }: BolumProps) {
  return (
    <section className="bolum">
      <h2>{baslik}</h2>
      {children}
    </section>
  );
}
```

Beklenen sonuç: Editörde hata yok; bu bileşeni de henüz kullanmadınız — bir sonraki bölümde `Koltuk` bileşenini yazdıktan sonra hepsini `App.tsx`'te birleştireceksiniz.

Kontrol Noktası — devam etmeden doğrulayın

`tipler.ts/veri.ts` dosyalarınız Hafta 3 ile birebir aynıysa, `style.css`'iniz devredeyse ve `EtkinlikKarti/ Bolum` bileşenleriniz editörde hatasız duruyorsa, props ve children temelini kurmuşsunuz demektir — artık bileşenlere bir hafıza (state) kazandırabilirsiniz.

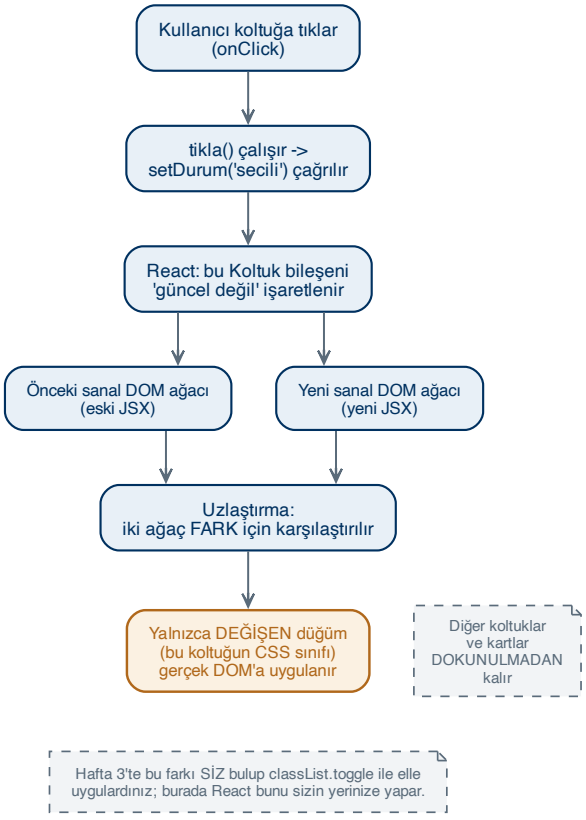
3 state ve Olaylar

(*bkz. Ders notu Hafta 4, Ünite 3 — state ve Olaylar.*)

Adım 8: `Koltuk` bileşenini `state` ve tıklama olayıyla yazın

Ne yapacağız? `src/bilesenler/Koltuk.tsx` dosyasını oluşturup `koltuk: Koltuk` alanı taşıyan `KoltukProps` arayüzünü, `useState<KoltukDurumu>(koltuk.durum)` ile yerel `durum` state'ini, `durum === 'dolu'` ise erken çıkan `tikla` fonksiyonunu ve `onClick/disabled/aria-disabled` özniteliklerini kullanan render kısmını yazacaksınız.

Neden? `useState`, bir bileşenin render'lar arasında hatırlanan kendi verisini (durumunu) tutabilmesini sağlayan bir React "hook"udur — `const [durum, setDurum] = useState<KoltukDurumu>(...)` çağrısı iki eleman döndürür: mevcut değer ve onu değiştirmenin tek yolu olan güncelleme fonksiyonu. Bu `useState` çağrısı yalnızca `Koltuk` bileşenine aittir — `App` ya da `KoltukPlani` bu değeri doğrudan göremez, bu haftanın bilinçli sınırıdır. C++'taki `const` bir değişkeni değiştirememe disiplinine benzer şekilde, state'i güncellemenin tek yolu `setDurum`'u çağırmaktır; `disabled/aria-disabled`, `dolu` bir koltuğun hem tıklanamaz hem ekran okuyucular için erişilebilir olmasını sağlar.




```tsx
interface KoltukProps {
  koltuk: KoltukTipi;
}
```

```tsx
// Koltuğun kendi durumu, koltuk.durum başlangıç değeriyle yerel bir
// useState'te tutulur; App ya da KoltukPlani bu değeri GÖRMEZ.
const [durum, setDurum] = useState<KoltukDurumu>(koltuk.durum);

function tikla() {
  if (durum === 'dolu') return;
  setDurum((onceki) => (onceki === 'bos' ? 'secili' : 'bos'));
```

Adım 8 (devamı)

```
}
```

```
return (
  <button
    type="button"
    className={`koltuk ${durum}`}
    onClick={tikla}
    disabled={durum === 'dolu'}
    aria-disabled={durum === 'dolu'}
  >
    {koltukEtiketi(koltuk)}
    {durum === 'dolu' && <span className="koltuk-rozet">Dolu</span>}
  </button>
);
```

Beklenen sonuç: Editörde hata yok. Kodu okuyarak `dolu` durumundaki bir koltuğa tıklamanın `tikla` fonksiyonu içindeki erken `return` ile engellendiğini görebilirsiniz — bu bileşeni tarayıcıda birazdan `App.tsx`'e bağladığınızda gerçekten test edeceksiniz.

Adım 9: Değişmezliği bozmayı deneyin (bilerek hatalı bir atama yapıp derleyiciyi gözlemleyin)

Ne yapacağız? `Koltuk.tsx`'te, yalnızca bu deney için, `useState` satırının hemen altına geçici bir satır — `durum = 'secili';` — ekleyeceksiniz. Ardından `npm run build` komutunu çalıştırıp derleyici hatasını gözlemleyecek, deneyi bitirince bu geçici satırı sileceksiniz.

Neden? `useState`'in döndürdüğü ilk eleman `const` bağlamlı bir değişkendir — doğrudan atama yapılamaz. State'i değiştirmenin tek yolu ikinci elemanı (`setDurum`) çağırmaktır; bu çağrı React'e "bu bileşeni yeniden render et" sinyalini verir, doğrudan atama ise React'e hiçbir şey söylemez, ekran değişmez. JavaScript'te bu hata sessizce geçerdi; TypeScript bu hatayı siz kodu yazarken, derleme sırasında yakalar.

Terminal

```
$ npm run build
```

Beklenen sonuç: Derleyici, geçici satırın bulunduğu satırda "sabit bir değişkene atama yapılamaz" anlamına gelen bir hata verir; aşağıdaki satır numaraları sizin dosyanızdaki tam konuma göre değişebilir — önemli olan hata metnidir. Geçici satırı sildikten sonra `npm run build` tekrar hatasız biter.

Beklenen Çıktı

```
src/bilesenler/Koltuk.tsx(22,3): error TS2588: Cannot assign to
'durum' because it is a constant.
src/bilesenler/Koltuk.tsx(36,17): error TS2367: This comparison
appears to be unintentional because the types '"secili"' and
'"dolu"' have no overlap.
```

Adım 9 (devamı)

> **İpucu**
>
> İkinci satırdaki gibi ek hatalar da görebilirsiniz — bunlar, TypeScript'in ilk (geçersiz) atamadan sonra `durum`'un tipini tek bir sabit değere daraltmaya çalışmasının yan etkisidir. Asıl önemli olan ilk satırdaki "`Cannot assign to ... because it is a constant`" mesajıdır.

Kontrol Noktası — devam etmeden doğrulayın

`Koltuk` bileşeniniz `useState` ile kendi durumunu tutuyor, `dolu` koltukta `tikla` erken dönüş yapıyor ve doğrudan atama denemenizde derleyici hata verdiyse, state ve değişmezlik ilkesini doğru kavramışsınız demektir — artık listeleri üretmeye geçebilirsiniz.

4 Liste ve Koşullu Render

(bkz. Ders notu Hafta 4, Ünite 4 — Liste ve Koşullu Render.)

Adım 10: `EtkinlikListesi` bileşenini `map` ve `key` ile yazın

Ne yapacağız? `src/bilesenler/EtkinlikListesi.tsx` dosyasını oluşturup `etkinlikler: Etkinlik[]` alanı taşıyan `EtkinlikListesiProps` arayüzünü, boş dizi için erken dönüşü ve `etkinlikler.map(...)` ile her etkinlik için bir `EtkinlikKarti` üreten (her birine `key={etkinlik.id}` vererek) gövdeyi yazacaksınız.

Neden? Hafta 3'te her kartı `forEach` + `createElement` ile DOM'a elle basıyordunuz; `.map(...)` bunun bildirimsel karşılığıdır — bir diziden doğrudan bir JSX öğe dizisi üretir. `key`, listedeki her öğeye veriden gelen kararlı ve benzersiz bir kimlik vermenizi sağlar; React'in uzlaştırma sırasında "bu hangi öğeydi, hangisi değişti" sorusunu doğru cevaplaması için gereklidir (birazdan bunu bozup gözlemleyeceksiniz). Boş dizi durumu (`etkinlikler.length === 0`) bir erken dönüşle (`if (...) return <...>;`) geri kalan JSX'ten önce ayrı ele alınır.

```ts
interface EtkinlikListesiProps {
  etkinlikler: Etkinlik[];
}
```

```ts
// Erken dönüş: boş dizi özel bir durumdur, geri kalan JSX'ten önce
// ayrı ele alınır.
if (etkinlikler.length === 0) {
  return <p>Henüz etkinlik yok.</p>;
}
```

```ts
// map ile diziden bileşen listesi üretme; her karta veriden gelen
// kararlı bir key (etkinlik.id) verilir.
return (
  <div className="etkinlik-listesi">
    {etkinlikler.map((etkinlik) => (
      <EtkinlikKarti key={etkinlik.id} etkinlik={etkinlik} />
    ))}
  </div>
);
```

Beklenen sonuç: Editörde hata yok; bu bileşen henüz `App.tsx`'te kullanılmıyor.

Adım 11: `KoltukPlani` bileşenini `map` ve `key` ile yazın

Ne yapacağız? `src/bilesenler/KoltukPlani.tsx` dosyasını oluşturup `koltuklar: Koltuk[]` alanı taşıyan `KoltukPlaniProps` arayüzünü, boş dizi için üçlü operatörle mesaj gösteren ve `koltuklar.map(...)` ile `Koltuk` ızgarasını üreten gövdeyi yazacaksınız.

Neden? Aynı map+key kalıbının ikinci bir uygulamasıdır — burada boş durum bir erken dönüş yerine üçlü operatörle (`? :`) ele alınır, çünkü iki alternatif (mesaj / ızgara) zaten birbirine yakın ve tek bir `return` ifadesi içinde okunaklı kalır; erken dönüş tercih etmek de yanlış olmazdı — ikisi de geçerli koşullu render kalıplarıdır.

```
interface KoltukPlaniProps {
  koltuklar: KoltukTipi[];
}
```

```
// Boş dizi için üçlü operatörle mesaj; aksi halde map+key ile Koltuk
// ızgarası üretilir.
export function KoltukPlani({ koltuklar }: KoltukPlaniProps) {
  return koltuklar.length === 0 ? (
    <p>Bu etkinlik için koltuk tanımlı değil.</p>
  ) : (
    <div className="koltuk-plani">
      {koltuklar.map((koltuk) => (
        <Koltuk key={koltuk.id} koltuk={koltuk} />
      ))}
    </div>
  );
}
```

Beklenen sonuç: Editörde hata yok; bu bileşen henüz `App.tsx`'te kullanılmıyor.

Adım 12: `App.tsx` ve `main.tsx` ile tüm bileşenleri birleştirip tarayıcıda test edin

Ne yapacağız? `src/App.tsx` dosyasının içeriğini silip, `veri.ts`'ten gelen `etkinlikler/koltuklar` dizilerini, `Bolum` ile sarmalanmış iki bölümde `EtkinlikListesi`/`KoltukPlani`'ya `props` olarak geçiren `App` bileşenini yazacaksınız. Ardından `src/main.tsx` dosyasını, `createRoot` ile `App` bileşenini `StrictMode` içinde `#root`'a bağlayacak biçimde güncelleyeceksiniz (`style.css` içe aktarmasını zaten Adım 5'te eklemiştiniz, yalnızca `App`'i içe aktarıp render eden kısım değişecek).

Neden? `App`, bileşen ağacının köküdür; her alt bileşen kendi işini yapan bağımsız bir birimdir, `App` onları bir araya getirir — Ünite 2'nin bileşen ağacı ve tek yönlü veri akışı fikri burada somutlaşır: `props` yalnızca `App`'ten aşağıya akar, hiçbir alt bileşen kendi verisini üst bileşene doğrudan göndermez.

```
// createRoot ile App bileşenini #root'a, StrictMode içinde bağlar.
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

```
// veri.ts'ten gelen sabit dizileri, Bolum (children) ile sarmalanmış iki
// bölümde EtkinlikListesi/KoltukPlani'ya prop olarak geçirir.
export function App() {
  return (
    <>
      <Bolum baslik="Etkinlikler">
```

Adım 12 (devamı)

```
      <EtkinlikListesi etkinlikler={etkinlikler} />
    </Bolum>
    <Bolum baslik="Koltuk Planı">
      <KoltukPlani koltuklar={koltuklar} />
    </Bolum>
  </>
 );
}
```

```
>_ Terminal

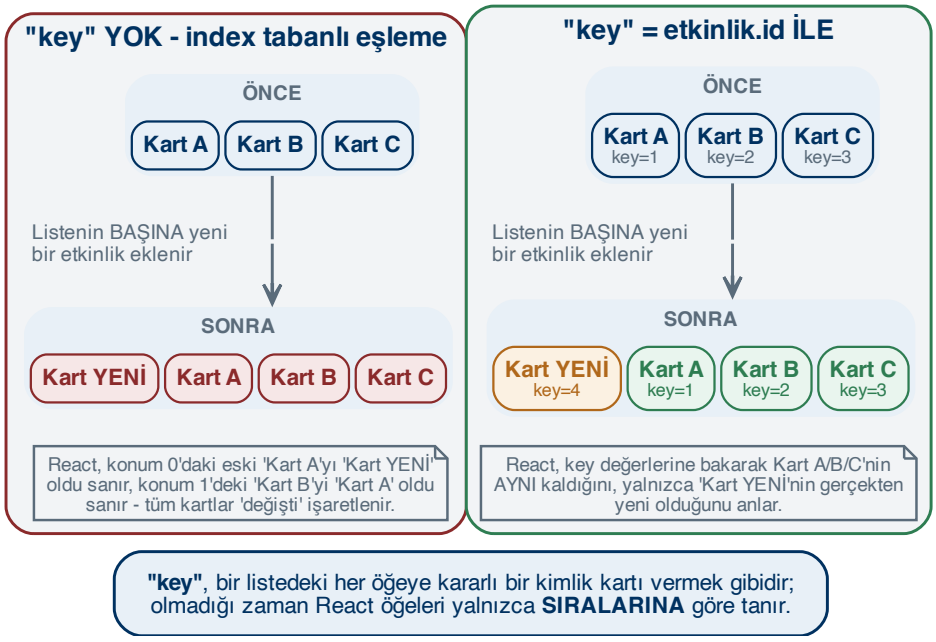
$ npm run dev
```

Beklenen sonuç: Tarayıcıda iki `Bolum` başlıklı bölüm görünür — "Etkinlikler" bölümünde 3 etkinlik kartı (Kariyer Günleri'nde "Ücretsiz" rozeti), "Koltuk Planı" bölümünde 3×4'lük bir ızgara. Boş bir koltuğa tıkladığınızda rengi "seçili" durumuna döner, tekrar tıklayınca geri döner; `dolu` işaretli iki koltuk (B2, C4) tıklanamaz ve üzerlerinde "Dolu" rozeti görünür. Bir koltuğu seçmenin başka hiçbir koltuğu ya da bir toplam sayacı etkilemediğini fark edin — bu, bu haftanın bilinçli sınırıdır (Hf.5 bunu bir üst bileşene taşıyacaktır).

Adım 13: `key` deneyi: `key`'i kaldırıp React'in uyarısını gözlemleyin

Ne yapacağız? `EtkinlikListesi.tsx`'teki `key={etkinlik.id}` satırını geçici olarak silin (ya da `key={Math.random()}` yapın), tarayıcıyı yenileyip DevTools konsolunu (F12 → Console) açarak React'in bastığı uyarıyı okuyun; ardından `key={etkinlik.id}`'ye geri dönün.

Neden? `key` olmadan (ya da dizideki sıraya/`index`'e dayalı bir key ile) React listedeki öğeleri yalnızca sıralarına göre eşlemeye çalışır — listenin başına yeni bir öğe eklendiğinde React tüm öğeleri yanlış eşleyip gereksiz yere yeniden oluşturabilir. `Math.random()` kullanıldığında ise her render'da farklı bir `key` üretileceğinden React o öğeyi her seferinde sıfırdan oluşturulmuş sanır. Kararlı ve benzersiz bir key (burada `etkinlik.id`), React'in "bu hangi öğeydi, hangisi değişti" sorusunu doğru cevaplamasını sağlar.



Adım 13 (devamı)

Beklenen sonuç: Tarayıcı konsolunda aşağıdakine benzer bir uyarı görürsünüz — bu bir derleme hatası değildir, kod yine çalışır, ama React sizi bilgilendirir.

Beklenen Çıktı

```
Warning: Each child in a list should have a unique "key" prop.

Check the render method of `EtkinlikListesi`. See
https://react.dev/link/warning-keys for more information.
```

Dikkat

`key` olarak dizideki `index`'i (`0, 1, 2, ...`) kullanmak da benzer bir soruna yol açar — bir `index` sıraya bağlıdır, veriye bağlı değildir; dolayısıyla kararlı bir kimlik sayılmaz. `key` her zaman veriden gelen, o öğeye özgü bir alan (burada `etkinlik.id`) olmalıdır.

Kontrol Noktası — devam etmeden doğrulayın

`EtkinlikListesi` ve `KoltukPlani` bileşenlerinizi `App.tsx`'te birleştirdiyseniz, tarayıcıda 3 kart ve 3×4 ızgarayı gördüyseniz ve `key` deneyinde React'in konsol uyarısını gözlemlediyseniz, bu haftanın dört ünitesini de uygulamışsınız demektir.

Adım 14: Kendiniz ekleyin: koltuk ızgarasına yeni bir sıra ekleyin

Ne yapacağız? `veri.ts`'teki `koltuklar` dizisine, 4. sıra için dört yeni `Koltuk` nesnesi (`sira: 4`, `koltukNo: 1..4`, hepsi `durum: 'bos'`, `id` değerleri 13–16) elle ekleyin. Ardından `style.css`'teki `.koltuk-plani` kuralındaki `grid-template-rows` değerini `repeat(3, 1fr)`'den `repeat(4, 1fr)`'e güncelleyin.

Neden? Bu görev, Ünite 4'ün `map+key` kalıbının gücünü elle deneyimlemenizi sağlar — `KoltukPlani` bileşenine tek satır bile dokunmadan, yalnızca veriyi değiştirerek ızgara büyür. Hafta 3'te aynı işi `koltukPlaniUret` fonksiyonunun döngüsünü yeniden çalıştırarak yapıyordunuz, sonuç benzerdi ama kod'a dokunmanız gerekiyordu; bu hafta yalnızca veri değişti.

Beklenen sonuç: Tarayıcıyı yenilediğinizde koltuk planı artık 4 sıra × 4 sütun (16 koltuk) görünür; yeni eklediğiniz 4. sıradaki koltuklar `'bos'` olduğundan tıklanabilir olmalıdır.

İpucu

Yeni `id` değerlerinin mevcut 12 koltuktan (1–12) farklı olduğundan emin olun — aynı `id`'yi tekrar kullanmak, iki farklı koltuğun aynı `key`'i paylaşmasına, dolayısıyla React'in uzlaştırma sırasında ikisini karıştırmasına yol açar.

Kontrol Noktası — devam etmeden doğrulayın

Koltuk planının 4×4 ızgara gösterdiğini ve yeni sıradaki koltukların tıklanabilir olduğunu doğruladıysanız, bu haftanın React katmanı işlevsel olarak tamamlanmış demektir — son adımda derlenebilirliğini doğrulayacaksınız.

Adım 15: Derlenebilirlik kapısını çalıştırın

Ne yapacağız? `istemci-react` klasöründe sırasıyla `npm ci` (ya da `npm install`), `npx tsc -b` ve `npm run build` komutlarını art arda çalıştıracaksınız.

Neden? Bu üç komut, bu haftanın "kapısıdır" — `npm ci` bağımlılıkların `package-lock.json`'a birebir sadık kurulduğunu, `npm run build` ise yazdığınız tüm TypeScript/TSX kodunun (bileşenler, props tipleri, state) tutarlı olduğunu ve projenin gerçek bir sunucuya yüklenebilecek durumda olduğunu garanti eder.

Terminal

```
$ npm ci
$ npx tsc -b
$ npm run build
```

Beklenen sonuç: Üç komut da hatasız biter; son komut `dist/index.html` ve `dist/assets/` altında CSS/JS dosyalarını üretir.

Beklenen Çıktı

```
added 27 packages, and audited 28 packages in 1s

found 0 vulnerabilities

vite v8.2.2 building client environment for production...
transforming...
✓ 22 modules transformed.
dist/index.html                   0.47 kB │ gzip:  0.31 kB
dist/assets/index-MSg5mKUT.css    1.68 kB │ gzip:  0.69 kB
dist/assets/index-jJ6tGDJU.js   192.80 kB │ gzip: 60.75 kB

✓ built in 507ms
```

İpucu

Hafta 3'te `npx tsc --noEmit` yazıyordunuz; bu haftadan itibaren `-b` (*build* modu) bayrağı zorunludur. `react-ts` şablonunun kök `tsconfig.json`'ı yalnızca proje referansları içerir (`"files": []`); bayraksız `npx tsc --noEmit` bu yüzden hiçbir şey denetlemez, her zaman sessizce "geçer". `npx tsc -b` ise referans projelerini (`tsconfig.app.json`, `tsconfig.node.json`) gerçekten derleyip tip denetimi yapar; `npm run build`'in ilk adımı da aynı komuttur — Adım 9'daki derleyici hatasını üreten de bu adımdı. Bir tip hatası varsa zincir `npx tsc -b` adımında durur.

Sorun Giderme

Belirti	Olası neden	Çözüm
`npm create vite@latest` React şablonu yerine eski bir liste gösteriyor	Eski bir `create-vite` sürümü önbelleğe alınmış olabilir	`npm create vite@latest -- --template react-ts` ile şablonu doğrudan komut satırından belirtin
Tarayıcıda beyaz boş sayfa, konsolda "Failed to resolve import" hatası	İçe aktarma yolunda dosya adı/uzantı yanlış yazılmış	İçe aktarma satırındaki dosya adının ve klasör yolunun (ör. `./bilesenler/Koltuk.tsx`) birebir doğru olduğunu kontrol edin
"Cannot find module react" hatası	`node_modules` eksik, `npm install` hiç çalıştırılmamış	`istemci-react` klasöründe `npm install` (ya da `npm ci`) çalıştırın
Bileşen ekranda hiç görünmüyor, konsolda hata yok	Bileşen fonksiyonu `App.tsx`'te hiç çağrılmamış/içe aktarılmamış	`App.tsx`'teki içe aktarma satırını ve JSX çağrısını kontrol edin
Bileşen HTML etiketi gibi davranıyor, sürekli boş kutu görünüyor	Bileşen adı küçük harfle başlıyor (ör. `etkinlikKarti`)	Bileşen adını büyük harfle başlatın (PascalCase): `EtkinlikKarti`
Koltuğa tıklayınca hiçbir şey olmuyor, konsolda hata yok	`onClick` yerine küçük harfle `onclick` yazılmış ya da `tikla()` çağrılarak (parantezle) referans verilmiş	JSX özniteliğinin `onClick` (camelCase) olduğunu ve `onClick={tikla}` biçiminde — parantez olmadan — referans verildiğini kontrol edin
`npm run build` "Type ... is not assignable" hatasıyla duruyor	`veri.ts`'teki bir nesnenin bir alanı eksik ya da yanlış tipte yazılmış	Hata mesajındaki satır numarasına gidip alan adını ve değerini `tipler.ts`'teki tanımla birebir karşılaştırın

5 · İzleme — Aktarım — Başka Nerede?

Bugün Hafta 3'ün `Etkinlik`/`Koltuk`/`Bilet` tipleri ve sabit veri dizileri değişmeden, artık React bileşenlerinin `props` tipleri olarak Etkinlik/Bilet Sisteminin ön yüzünü oluşturuyor — `EtkinlikKarti`, `EtkinlikListesi`, `Koltuk`, `KoltukPlani` bugünden itibaren dönem boyunca sizinle kalacak. Hf. 5'te bu haftanın bilinçli bıraktığı boşluğu — koltuk seçiminin yalnızca `Koltuk` bileşeninin kendi içinde yaşaması — durum kaldırma (*lifting state up*) ile dolduracak, bir koltuk seçme formu ve gerçek bir sepet oluşturacaksınız; `useEffect`, kontrollü form girdisi ve `Context` bu köprüyü kuracaktır. Hf. 6'da bugün sabit (mock) olan `veri.ts` dizileri yerini FastAPI'den `fetch` ile gelen gerçek veriye bırakacak — `EtkinlikKarti`/`Koltuk` bileşenlerinin `props` tipleri aynen kalacak, yalnızca verinin kaynağı değişecek. Hf. 7'de tek `App` bileşeniniz React Router ile birden çok sayfaya bölünecek. Bugün öğrendiğiniz `map`+`key` kalıbı, ilerleyen haftalarda büyüyecek her listede (etkinlik arama sonuçları, kullanıcının biletleri) aynen kullanılacaktır.